
A BUILD SYSTEM FOR DEVELOPERS · CLAUDE CODE · 2026

The 2-Minute Website Stack

The exact workflow — Claude Code, a UI/UX skill, 21st.dev components and Motion animation — packaged into one repository and one command.

4

tools combined into
one repeatable workflow

1

command to start a new
premium site from scratch

0

times you copy-paste the
same setup steps again

Sanchit Pandey

AI systems, automation & web development
Lucknow, India

It is not one trick, it is a stack

"AI built a website in 2 minutes" is technically true and a little misleading. What actually happened is four tools doing what each is good at, wired together so you never re-do the wiring: an agent that can read, write and run code in your project; a source of real, premium-quality components instead of generic ones; a motion library so it does not feel static; and a written-down standard for what "premium" means, so the agent applies it the same way every time.

The two minutes is real once the setup exists. This document is the setup — built once, then reused for every site after.

WHAT THIS IS NOT

Not a no-code tool and not a magic prompt. This produces a real Next.js codebase you own, can read, and can hand to any developer. The skill is real engineering instructions; the speed comes from never re-explaining them.

What you need before starting

- A terminal, and comfort running a handful of commands — this is a developer workflow, not a point-and-click tool
- Node.js installed, and a Claude subscription (Pro, Max, Team or Enterprise) or Console account
- A free 21st.dev account for component access
- About an hour for the one-time setup in this document. After that, each new site really does start in minutes

The four pieces

Understanding what each layer actually does is what lets you debug it later, and what lets you swap a piece out without breaking the rest.

1

CLAUDE CODE — The agent

Reads your repo, writes files, runs commands, and holds a memory file (CLAUDE.md) that persists project context across sessions

2

A UI/UX SKILL — The standard

A folder of written instructions — design principles, spacing rules, what "premium" means — that Claude loads automatically when the task calls for it

3

21ST.DEV — The components

A registry of real, production-grade React/Tailwind components. Claude copies and adapts these instead of generating generic UI from scratch

4

MOTION — The animation

The React animation library formerly called Framer Motion — handles entrance transitions, scroll effects and gestures without hand-written animation code

A naming note

Framer Motion was renamed Motion in 2025 when it became an independent project. The API is unchanged; the package moved from `framer-motion` to `motion` on npm, and the import path is now `motion/react`. Most people still say "Framer Motion" out of habit — this document uses Motion, since that is what you will actually install.

How Claude Code skills actually work

Worth understanding properly, because the entire workflow depends on it and most explanations online are vague about the mechanics.

The model

- A skill is a folder containing a `SKILL.md` file — instructions, conventions and, optionally, supporting files or scripts
- Personal skills live in `~/.claude/skills/` and apply across every project on your machine
- Project skills live in `.claude/skills/` inside a repo and travel with it — anyone who clones the repo gets the skill
- Claude reads the skill's `description` field to decide when it is relevant, and loads the full instructions only when a task actually calls for it — this is why description quality matters more than most people expect
- A skill can be invoked automatically when Claude judges it relevant, or manually by typing `/skill-name`

Skills vs. the project memory file

	CLAUDE.md	A skill
What it is	One file of standing project context	A folder, possibly with scripts and reference files
Scope	Always loaded for the project	Loaded only when the task matches its description
Best for	"This repo uses Tailwind, the API lives in <code>/api</code> , run tests with <code>npm test</code> "	A repeatable skill — design standards, a review process, a way of building one kind of thing

Treat `CLAUDE.md` as what Claude should always know about this project, and a skill as what Claude should know how to do — in this case, build premium UI — regardless of which project it is working in.

This section reflects the skills mechanism as documented in mid-2026. Claude Code changes quickly — check code.claude.com/docs/en/skills for anything that looks different by the time you read this.

The manual workflow first

Run this by hand once before automating it. You need to see each step work on its own before you trust a script to do it for you.

Step 1 — Install Claude Code

TERMINAL

```
npm install -g @anthropic-ai/claude-code
claude --version

# Run inside your project folder to start a session:
claude
```

Step 2 — Create a new project

TERMINAL

```
npx create-next-app@latest my-premium-site --typescript --tailwind --app
cd my-premium-site
```

Step 3 — Add Motion

TERMINAL

```
npm install motion
# import in a component:
# import { motion } from "motion/react"
```

Step 4 — Pull a component from 21st.dev

Browse 21st.dev, find a component, and either copy its install command directly, or copy its "AI prompt" and hand it to Claude Code — the prompt tells the agent exactly what to fetch and how to wire it into your stack.

TERMINAL

```
# 21st.dev gives you a ready CLI install for most components, e.g.:
npx shadcn@latest add "https://21st.dev/r/[author]/[component-name]"
```

Step 5 — Start the Claude Code session and build

With the project scaffolded and the component installed, this is where you actually describe the site to the agent.

Writing the UI/UX skill

This is the piece that makes output consistently premium instead of consistently generic. Without it, the agent has good taste on average and no taste in particular — it needs your standard written down.

Where it lives

TERMINAL

```
mkdir -p ~/.claude/skills/premium-ui-ux
touch ~/.claude/skills/premium-ui-ux/SKILL.md

# Personal skill: every project on this machine gets it.
# For a team-shared version, use .claude/skills/ inside the
# repo instead, so it travels with git.
```

A working SKILL.md

Adapt the specifics — this is a real starting point, not a placeholder. Split across two blocks here for space; in your repo it is one continuous file.

SKILL.md — PART 1 of 2

```
---
name: premium-ui-ux
description: Use when building or styling any web page, landing
page, or UI component. Applies premium visual design standards
- spacing, typography, motion, and layout conventions - so
output looks designed rather than default. Trigger on any
request to build, style, redesign, or polish a website, page,
or component.
---

# Premium UI/UX Standard

## Typography
- One display font for headings, one system/sans font for body.
Never more than two font families on a page.
- Type scale: use a real ratio (1.25 or 1.333), not arbitrary
pixel sizes. Headings should look like they belong to a system.
- Line length for body text: 60-75 characters. Line height 1.5-1.7
for body, 1.1-1.25 for large headings.

## Spacing
- Use a spacing scale (4px base: 4, 8, 12, 16, 24, 32, 48, 64,
96). Never an arbitrary value like 13px or 22px.
- Sections need real breathing room: 80-160px vertical padding on
desktop, scaled down on mobile - not 24px between every block.
- Group related elements tighter than unrelated ones. Spacing
should communicate hierarchy before color does.

## Color
- One accent color used deliberately, not scattered. Everything
else is neutral (near-black, near-white, a small grey ramp).
- Check contrast: body text at minimum 4.5:1 against its
background. Do not guess - state the ratio you are targeting.
- Dark mode is not inverted colors. Define both palettes
intentionally if both are required.
```

SKILL.md, part 2

PROMPT

```
## Motion (using the `motion` package, import from "motion/react")  
- Entrance animations: fade + 8-16px translate, 300-500ms,  
ease-out. Nothing longer than 600ms for a UI transition.  
- Stagger related items (a card grid, a list) by 40-80ms each -  
do not animate everything simultaneously.  
- Scroll-triggered reveals use `whileInView`, trigger once, and  
respect `useReducedMotion` for accessibility.  
- Motion should clarify relationships (what appeared because of  
what) - never add animation with no informational purpose.
```

```
## Layout  
- Every section needs a clear single job. If you cannot name it  
in four words, split the section.  
- Asymmetry and intentional whitespace over centering everything.  
A centered page with equal margins everywhere reads as default,  
not designed.  
- Mobile is not a shrunk desktop layout. Design the mobile stack  
order deliberately - the most important element first.
```

```
## Components  
- Prefer adapting a real component from 21st.dev over generating  
one from scratch. Match its structure and Tailwind conventions  
to the rest of the codebase rather than leaving it as a visual  
outlier.  
- Every interactive element needs a hover, focus and active state  
- not just a default state.
```

```
## Before finishing any UI task  
- Check spacing against the scale above.  
- Check that motion durations are in range.  
- Check color contrast on body text.  
- Check that the mobile layout was considered, not just scaled.
```


Turning it into one repository

This is the actual point of the exercise — not repeating the four setup steps for every new site. A starter repository with everything pre-wired, plus one setup script.

What the repository contains

REPOSITORY STRUCTURE

```
premium-site-starter/  
  .claude/  
  skills/  
  premium-ui-ux/  
  SKILL.md <- the skill from Section 04, project-scoped  
  CLAUDE.md <- project conventions, see below  
  app/  
  layout.tsx  
  page.tsx  
  components/  
  ui/ <- 21st.dev components land here  
  lib/  
  motion-variants.ts <- shared animation presets  
  setup.sh <- the one command  
  package.json  
  tailwind.config.ts
```

A starter CLAUDE.md

CLAUDE.md — PROJECT ROOT

```
# Project conventions
```

```
Stack: Next.js (App Router), TypeScript, Tailwind, Motion  
(`motion/react`), shadcn/ui component patterns.
```

- New UI work should follow the premium-ui-ux skill in .claude/skills/. Do not skip it for "quick" components.
- Prefer components adapted from 21st.dev over generating from scratch. Ask before installing a new dependency.
- All animation goes through lib/motion-variants.ts - add new variants there rather than inlining transition objects.
- Mobile-first: build and check the mobile layout before the desktop one.
- Every new page needs a loading state and an empty state, not just the happy path.

The one-command setup script

This is what turns "install Claude Code, add the skill, pull a component, add Motion" into a single command. Save it as `setup.sh` at the root of your starter repository.

setup.sh

```
#!/bin/bash
set -e

echo "Setting up premium site starter..."

# 1. Install dependencies
npm install
npm install motion

# 2. Confirm Claude Code is available
if ! command -v claude &> /dev/null; then
  echo "Installing Claude Code..."
  npm install -g @anthropic-ai/claude-code
fi

# 3. Ensure the project skill is in place
mkdir -p .claude/skills/premium-ui-ux
if [ ! -f .claude/skills/premium-ui-ux/SKILL.md ]; then
  echo "Skill missing - copying from template..."
  cp templates/premium-ui-ux.SKILL.md .claude/skills/premium-ui-ux/SKILL.md
fi

# 4. Launch Claude Code with a starting brief
echo "Setup complete. Launching Claude Code..."
claude "Read CLAUDE.md and the premium-ui-ux skill, then ask me
3 questions about the site I want before building anything."
```

Making it genuinely one command

- 01** Push this starter repository to your own GitHub.
- 02** Anyone — including you, six months from now — clones it and runs `chmod +x setup.sh && ./setup.sh`.
- 03** That is the "one command" the workflow promises: it installs dependencies, confirms Claude Code is present, makes sure the skill is in place, and opens a Claude Code session already briefed on the project.
- 04** For repeated use across many client sites, publish the starter as a degit template or a GitHub template repository so a new copy is one line: `npx degit you/premium-site-starter new-client-site`.

The script does not remove judgement — it removes repetition. You still make every design decision; you just never re-explain the standard or re-run the same five setup commands.

Prompting the build

With the stack in place, this is what you actually type. Three prompts that take a new site from brief to first draft.

The brief

PROMPT

I'm building a website for [BUSINESS / PROJECT]. It's a [TYPE OF BUSINESS] and the site needs to [PRIMARY GOAL - book appointments / generate leads / sell a product / present a portfolio].

Audience: [WHO]

Tone: [e.g. premium and minimal / bold and energetic / warm and approachable]

Reference sites I like: [1-3 URLs, and what you like about each]

Pages needed: [LIST]

Must include: [e.g. booking form, WhatsApp button, pricing table]

Follow the premium-ui-ux skill for all styling decisions. Before writing code, propose a page structure and a component list for the homepage, and confirm the plan with me first.

Pulling a specific component

PROMPT

Find a [COMPONENT TYPE - e.g. pricing table / hero section / testimonial carousel] on 21st.dev that fits a [TONE] site. Show me 2-3 options with their preview links before installing anything.

Once I pick one: adapt it to our color palette and spacing scale from the premium-ui-ux skill, wire it to real content instead of placeholder text, and add entrance animation using our lib/motion-variants.ts presets.

The polish pass

PROMPT

Review the homepage against the premium-ui-ux skill point by point. List anything that violates the spacing scale, the motion duration limits, or the contrast requirement - then fix each one and tell me what changed.

Also check: does every interactive element have a hover and focus state? Is the mobile layout deliberately ordered, or just the desktop layout shrunk?

What actually reads as "premium"

The polish-pass prompt from Section 07 is the one people skip, and the one that makes the biggest visible difference — run it before calling any page finished.

The word "premium" gets used a lot and defined rarely. These are the concrete, checkable things that separate a premium-feeling site from a template-feeling one.

- ☐ Generous whitespace — most default AI output under-spaces; premium sites over-space
- ☐ One accent color, used with restraint, not five competing colors
- ☐ A type scale that follows a real ratio, not arbitrary sizes chosen by eye
- ☐ Motion that is subtle and purposeful — under 500ms, nothing bouncing or spinning without reason
- ☐ Real content in every draft, not lorem ipsum — placeholder text hides layout problems
- ☐ A hero that states what the business does in the first sentence a visitor reads
- ☐ Every button and link has a visible hover state
- ☐ Images that are actually optimised, not just visually present
- ☐ A mobile layout that was designed, not shrunk
- ☐ Consistent corner radius, shadow depth and border treatment across every component

The tell-tale signs of unreviewed AI output

- ✗ Every section has identical vertical spacing, regardless of content
- ✗ Three or more accent colors competing for attention
- ✗ Icons from three different icon sets mixed on one page
- ✗ Animation on everything, including things that gain nothing from moving
- ✗ Centered text and centered layout on every single section
- ✗ Generic stock-photo energy even when real images were provided

Where this workflow breaks

Handled honestly, since "it just works" is not true of any AI coding workflow yet.

Problem	Why it happens	Fix
Skill does not trigger	The description field is vague or does not mention the trigger words you actually use	Rewrite the description with the exact phrases you type — "build", "style", "landing page"
Output looks generic despite the skill	The skill was loaded but the agent had a more specific instruction in the same prompt that overrode it	State explicitly: "follow the premium-ui-ux skill for this" in the prompt itself
21st.dev component clashes visually with the rest of the site	Components come from many different authors with different conventions	Always ask Claude to adapt colors, spacing and radius to your existing tokens after installing
Animations feel excessive	Default AI output tends to animate everything it touches	Cap durations and scope in the skill, and explicitly ask for a pass that removes unnecessary motion
Site works locally, breaks on deploy	Environment variables, image domains, or a build-only error not caught in dev mode	Run a production build locally before deploying: <code>npm run build</code>
Claude re-explains itself every session	CLAUDE.md is missing, too vague, or was never committed to the repo	Keep CLAUDE.md specific and commit it — it is project memory, not a suggestion

None of this makes the workflow unreliable — it makes it a real engineering tool with real failure modes, which is a different thing from a magic prompt that either works or does not.

Turning this into a service

A repeatable build system is a genuinely sellable asset, separate from any individual website you build with it.

How to package it

- 01** Price the outcome, not the two minutes. The client is buying a finished, premium site and your judgement in reviewing it — not literally two minutes of compute. Your speed is your margin.
- 02** Keep one starter repo per site category — a landing page starter, a booking-site starter, a portfolio starter — each with its own CLAUDE.md tuned to that category's real requirements.
- 03** Never ship the first draft. Always run the polish-pass prompt from Section 07 and a manual review against Section 08's checklist before a client sees anything.
- 04** Keep the skill improving. Every time you personally fix something the agent got wrong, add the rule to the skill. It should be stricter after each project, not the same file forever.
- 05** Be honest about what the client is buying if asked: a fast, disciplined build process — not a website with no human judgement in it anywhere.

The speed is real and it is also the least interesting part to sell. What is actually valuable to a client is that the same standard gets applied every time, because it is written down instead of remembered.

Notes & sources

Checked July–August 2026. Claude Code's skill system and the 21st.dev registry both move fast — verify commands against current docs before relying on them.

What the guide says	Basis
Claude Code skills are folders with a SKILL.md, loaded from ~/.claude/skills/ (personal) or .claude/skills/ (project)	Claude Code documentation, code.claude.com/docs/en/skills
A skill's description field determines when Claude invokes it automatically	Claude API and Claude Code skills documentation, platform.claude.com
Framer Motion was renamed Motion in 2025; package is now `motion`, import `motion/react`, API unchanged	Motion's own documentation, motion.dev , and multiple independent 2026 sources
21st.dev is a component registry (not a page builder) with CLI install and an AI-generated prompt per component, usable with Claude Code and other AI coding tools	21st.dev product documentation and independent reviews, 2025–26
CLAUDE.md functions as always-loaded project memory, distinct from on-demand skills	Claude Code documentation

The SKILL.md content in Section 04 and the setup.sh script in Section 06 are original working examples written for this guide, not copied from any specific published repository — adapt the specifics to your own design taste before using them.

THAT'S THE STACK

The speed was never the trick. The setup was.

Anyone can watch Claude Code build a page in a terminal and call it magic. What actually happened is an agent, a written design standard, a real component source and an animation library, wired together once so the wiring never has to happen again. That is not a replacement for a developer's judgement — it is what makes a developer's judgement apply consistently at speed.

Build the skill in Section 04 first. Everything after that is assembly.

Sanchit Pandey

AI systems, automation and web development.

If you refine the premium-ui-ux skill into something sharper, send it to me. The best version of this skill will always be the one that has shipped the most real sites.

© 2026 Sanchit Pandey. Share it freely — keep the attribution.